

Mini-Katran: High-Performance L4 Load Balancing with eBPF and XDP

CS 331 Computer Networks, 2026-27 Semester 1 · Project 16 · Team T020

Akshit Chhabra (24110026) · Bhavay Goyal (24110070) · Devarshi Savalgi (24110103) · Kunal Chandola (24110176)

11 September 2026

ABSTRACT. We built a small layer-4 load balancer modelled on Facebook's Katran. The forwarding logic is an eBPF program attached at the XDP hook, so it runs inside the network driver before the kernel's network stack sees the packet: it parses the Ethernet, IPv4 and TCP headers, keeps connection state in a BPF map, picks a backend with consistent hashing, rewrites the destination address, patches the IP and TCP checksums and sends the frame back out. A user-space tool configures it through BPF maps. We compared it with two conventional user-space load balancers on the same network-namespaces testbed. The XDP version forwards 18.5 Gbit/s with four streams against 12.0 for a TCP proxy, handles 1.8 times as many connections per second and adds 3 μ s to a request instead of 22.

1 Introduction

A load balancer takes traffic sent to one public address, the virtual IP (VIP), and spreads it over several backend servers. A layer-4 load balancer does this using only IP addresses and TCP ports, without looking at the application protocol. The usual way to write one is as a normal program, but by the time a packet reaches a program the kernel has already allocated its packet structure, run it through the IP and TCP layers and copied it to user memory. At 10 Gbit/s a full-size frame arrives about every 1.2 microseconds, and that per-packet kernel work, not the load-balancing logic, becomes the limit.

Katran [1], which Facebook open-sourced in 2018, avoids this by making the forwarding decision in an eBPF program at the XDP hook, which the network driver calls for every received frame before the kernel does anything else with it. The goal of this project was to understand that design by building a small version of it and measuring it against the traditional approach. Following the project brief, we set out to:

- implement a basic layer-4 load balancer in user space as a baseline;
- implement an XDP program that parses Ethernet, IPv4 and TCP headers, extracts 5-tuple flow identifiers and keeps connection state in eBPF maps;
- select backends with a simple consistent hashing scheme, rewrite the packet's destination, recompute the L3 and L4 checksums and forward entirely in kernel space;
- build a control plane that shares configuration and state between the kernel program and user space;
- benchmark the two on the same testbed.

The code, scripts and raw results are in the `code/` directory of the repository.

2 Background

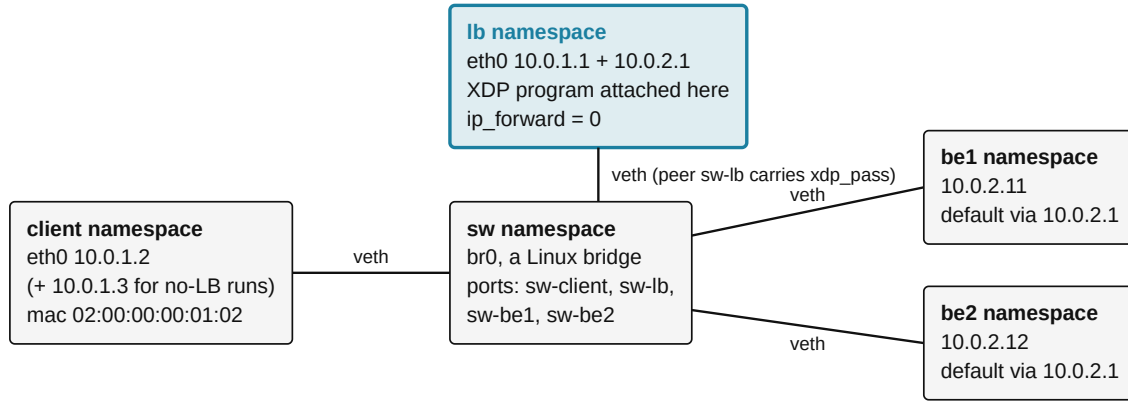
Layer-4 load balancing. Two things make it harder than picking a server at random. TCP state lives only at the two ends of a connection, so every packet of a connection must reach the same backend; a packet arriving at a machine with no matching socket gets a reset. And backends come and go, so the mapping from connections to backends should change as little as possible when the set of backends changes.

eBPF and XDP. eBPF lets you load small programs into the Linux kernel without writing a kernel module. The program is written in restricted C, compiled with clang to BPF bytecode and loaded through the `bpf()` system call. Before it may run, the verifier checks every path through it: every memory access must be provably in bounds, loops must be bounded, and only approved helper functions may be called. Programs talk to user space through maps, kernel data structures (arrays, hash tables, LRU hash tables) that both sides can read and write; a map can be pinned to a path in the BPF filesystem so it outlives the process that created it [4]. XDP (eXpress Data Path) is the hook in the network driver's receive path [2]. The program gets pointers to the raw frame and returns a verdict: `XDP_PASS` (continue into the normal stack), `XDP_DROP`, `XDP_TX` (send the frame back out of the same interface, possibly modified) or `XDP_REDIRECT`. In native mode the driver runs the program before allocating a socket buffer, and it runs once per receive queue, so it scales with queues and cores. The veth virtual interfaces we use support native XDP, which is what makes a single-machine testbed meaningful.

Katran. Katran's data plane looks each packet up in an LRU map, hashes new flows onto a backend table with Maglev hashing [3], and forwards to backends inside IP-in-IP tunnels so that backends reply to clients directly and the load balancer only sees the request direction. We kept the shape and simplified the parts: a hash ring instead of Maglev, destination rewriting instead of tunnels, IPv4 and TCP only.

3 Design

Testbed. Everything runs on one laptop in Linux network namespaces (Figure 1). The client is on 10.0.1.0/24, the two backends on 10.0.2.0/24, and the load balancer's single interface has an address on each. The VIP, 10.0.0.10, belongs to no interface: the client has a route for it via the load balancer, and the backends' default route points at the load balancer, so their replies pass through it too. The client and the backends are on different subnets on purpose: on one subnet a backend's reply would go straight to the client, arrive with the backend's address as source, and the client would reset the connection. IP forwarding is off in the load balancer namespace, so whatever the XDP program does not take is dropped. Offloads (TSO, GSO, GRO, checksum offload) are switched off on the veths so the load balancer sees real MTU-sized frames with real checksums, and each veth has eight queues, for the reason given in Section 5.



client route: 10.0.0.10/32 via 10.0.1.1. The VIP is on no interface; the LB intercepts it.
 backend routes: default via 10.0.2.1, so replies to clients pass through the LB again.
 lb: ip_forward = 0 and a blackhole route for 10.0.0.0/24. What XDP does not take is dropped.

Figure 1. The testbed built by `scripts/topo.sh`. The XDP program is attached to the load balancer's only interface.

Forwarding. For a packet from a client, the program rewrites the destination IP to the chosen backend and the destination MAC to the backend's, and sends it back out with `XDPTX` (Figure 2). The backend answers the client's address; because its gateway is the load balancer, the reply arrives at the same program, which rewrites the source back to the VIP and the destination MAC to the client's. Neither end learns the other's real address, and the load balancer holds no socket and no TCP state. Ports are not rewritten: backends listen on the VIP's port. We chose this over Katran's tunnelling because the brief asks for destination rewriting and checksum recomputation, which is exactly what tunnelling with direct server return avoids; the cost is that the load balancer handles both directions.

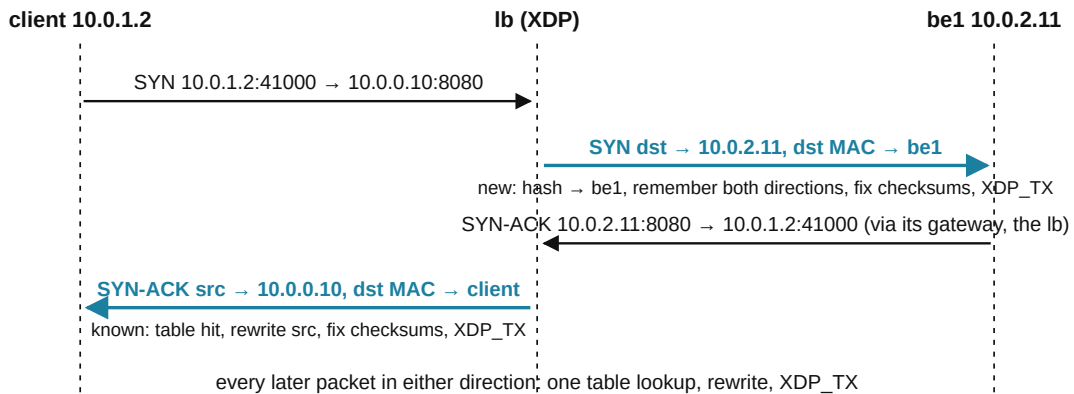


Figure 2. One TCP handshake through the load balancer.

Backend selection. `hash(client) mod n` would move almost every mapping whenever n changes. We use a hash ring: each backend gets 100 points on a 32-bit circle, the circle is divided into 65,537 slots, and each slot belongs to the first backend point clockwise of it. The ring is built in user space and written into a BPF array; the program does one array lookup with the hash of the client's address and port. With two backends the split is 51.9% to 48.1%, and adding a third moves 35.3% of the slots, close to the ideal third. A per-VIP flag leaves the client port out of the hash so that all connections from one client reach the same backend, which `iperf3` needs because it opens a control and a data connection.

Connection tracking. The ring answers where a new connection should go; the flows map answers where an existing connection already went, which keeps established connections in place when the ring changes. A new connection inserts two entries, one per direction, and the reverse entry stores the client's MAC so replies need no ARP. The map is an LRU hash: when full, the least recently used entry is evicted, so no expiry logic is needed. One rule sits on top, learned the hard way (Section 6): a TCP SYN never consults the table. It is hashed onto the ring and both entries are written fresh, so a lost or stale entry is harmless.

Control plane. A small libbpf program, `bin/lb`, loads the XDP program, pins its maps under `/sys/fs/bpf` and attaches it in native mode. Other commands add VIPs and backends, rebuild a VIP's ring, and dump the connection table and counters. It keeps no state of its own: the ring in the map is the only record of the configuration, so adding or removing a backend while traffic flows is one command.

Baselines. The TCP proxy is the classic design: a single-threaded epoll loop that accepts the client connection, connects to a backend chosen with the same ring, and copies bytes both ways. The `AF_PACKET` forwarder is the XDP logic moved to user space: it reads raw frames from a packet socket, applies the same lookup and rewrite against a table in process memory, and sends them back with `sendto()`. Between it and the XDP program the only difference is where the code runs.

4 Implementation

The XDP program, `bpf/lb_kern.c`, is about 200 lines of C. Every pointer into the frame is compared with `data_end` before it is used, because the verifier rejects any load it cannot prove in bounds; IP options, fragments and non-TCP traffic are passed to the stack, which keeps the headers at fixed offsets. After the flow lookup (or, for a new flow, the VIP, ring and backend lookups and the two map inserts) the program changes one IP address, both MACs and the two checksums. The IPv4 header checksum and the TCP checksum are both ones' complement sums, and the TCP one covers a pseudo-header containing the IP addresses, so an address change shifts both by the same amount. RFC 1624 [5] gives the update without touching the payload: when a field changes from m to m' , the new checksum is $\sim(\sim HC + \sim m + m')$:

```
static __always_inline __u16 csum_replace4(__u16 check, __be32 old, __be32 new)
{
    __u32 sum = (__u16)~check;
    sum += (__u16)~old + (__u16)(~old >> 16);
    sum += (__u16)new + (__u16)(new >> 16);
    sum = (sum & 0xffff) + (sum >> 16);
    sum = (sum & 0xffff) + (sum >> 16);
    return ~sum;
}
```

`tcpdump` on both sides of the load balancer reports every rewritten packet's checksum as correct, and a 20 MB file downloaded through the VIP has the same MD5 as the original. The verifier accepted the program on the first load; it compiles to 342 BPF instructions.

The control plane uses the plain libbpf API (`bpf_object__open_file`, `bpf_object__load`, `bpf_object__pin_maps`, `bpf_xdp_attach`). One detail cost us time: `ip netns exec` mounts a fresh `sysfs` in the namespace and hides `/sys/fs/bpf`, so pinning fails there; `nsenter --net` switches only the network namespace. Because the object carries BTF type information, `bpftool map dump` prints every map by field name, which we used throughout.

For benchmarking we wrote a tiny epoll HTTP server and a connection-rate generator, because Python's `http.server` manages only a couple of thousand requests per second and would have hidden any difference between load balancers. A smoke test brings the testbed up and checks, for each mode, that requests reach both backends, that a 10 MB download has the right MD5, and that iperf3 and the connection generator run.

5 Evaluation

Method. All measurements were taken on one laptop (Intel i7-14650HX, 24 logical cores, Linux 7.1) on the testbed above. Four modes were compared: *direct* (no load balancer, the load balancer namespace just routes), *proxy*, *raw* and *xdp*. The tests were iperf3 with one and with four streams, the connection generator with 64 connections in flight (connections per second) and with one (request latency), five seconds each, three repetitions with the modes interleaved, median reported. The CPU governor was pinned for the run. CPU cost is the whole machine's busy time from `/proc/stat`; for XDP we also read the program's own run time per packet from `kernel.bpf_stats_enabled`.

mode	1 stream Gbit/s	4 streams Gbit/s	conn/s (64 in flight)	request p50 µs	p99 µs (64)	machine CPU s/GB
direct (no LB)	13.1	39.3	86,136	30	1,343	1.84
TCP proxy	11.5	12.0	41,706	52	2,288	2.95
AF_PACKET forwarder	5.7	5.7	51,606	45	1,618	5.19
XDP	12.7	18.5	73,728	33	1,519	2.43

What the numbers say. The proxy is flat at 12 Gbit/s for one and four streams: it is one process on one core, at 75% of it. XDP does 1.5 times better with four streams because the work is spread over the receive queues. It is still below the no-LB figure because on veth every XDP_TX frame is copied twice and multi-stream traffic is reordered by the veth-and-bridge path, which costs XDP retransmissions that the proxy, which terminates TCP, does not see. For connection rate XDP handles 1.8 times the proxy's rate: the proxy runs two handshakes and holds two sockets per client connection. A request through XDP costs 3 µs more than with no load balancer; through the proxy 22 µs more. Subtracting the no-LB row, the load balancing itself costs 0.59 CPU-seconds per gigabyte in XDP, 1.11 in the proxy and 3.35 in the raw forwarder. The raw forwarder is worst for bulk traffic because a packet socket is a tap, not a steal: the kernel copies each frame to the socket and still runs it through its own stack, so the work is done twice. The XDP program itself takes 41 to 58 ns per packet for bulk traffic and about 250 ns for SYN-heavy traffic, where each new connection inserts two map entries.

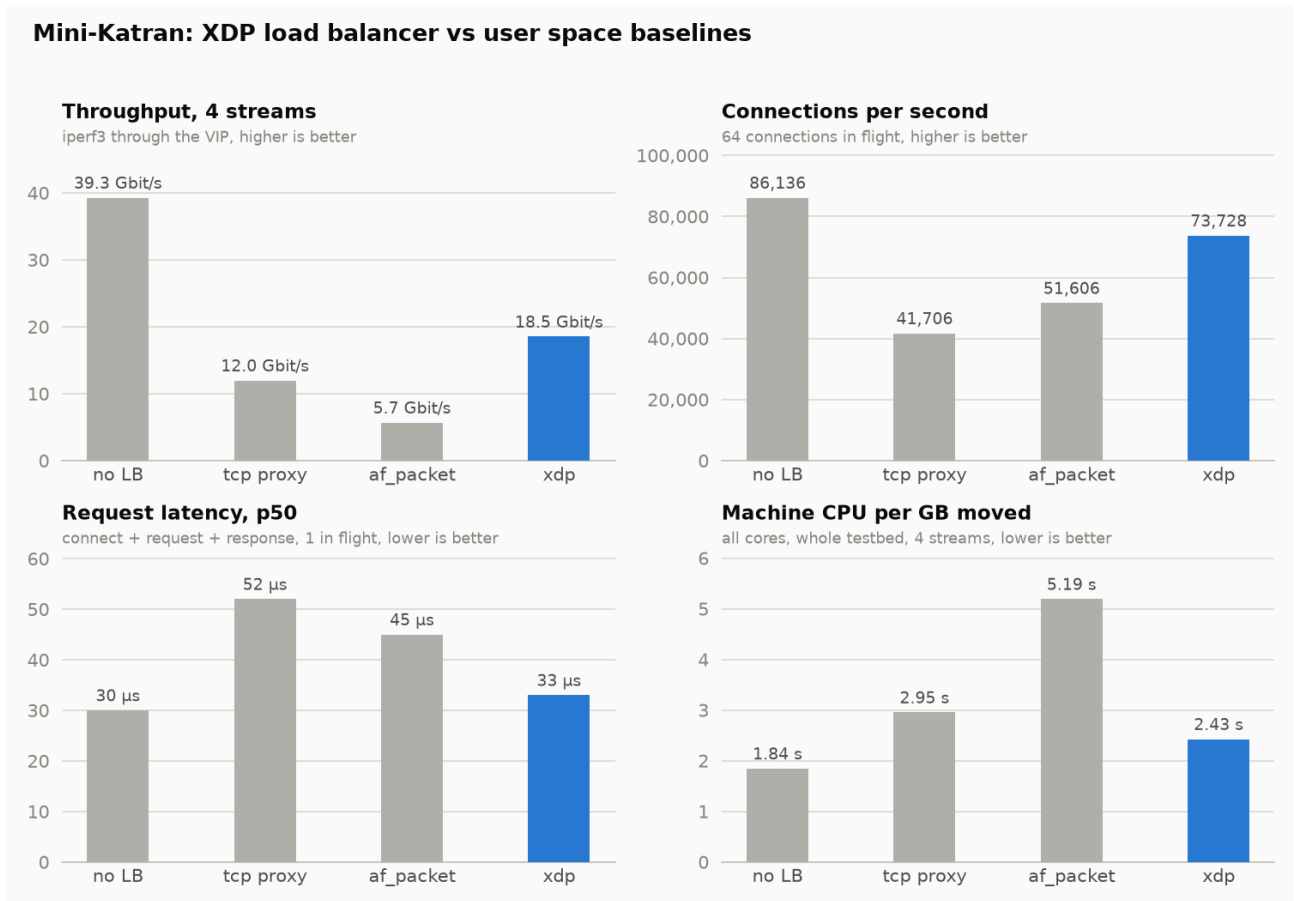


Figure 3. The four headline measurements, median of three repetitions.

Scaling. Our first benchmark round had XDP at the same 11.5 Gbit/s for one and four streams while the no-LB run reached 36. The program was using 5% of a core, so it was not the limit; `ethtool -S` showed every packet on `rx_queue_0`. The veths had one queue, so all XDP work ran on one CPU, like a NIC with receive-side scaling off. With eight queues per veth, four streams reached 18.5 Gbit/s and eight streams 29 (Figure 4).

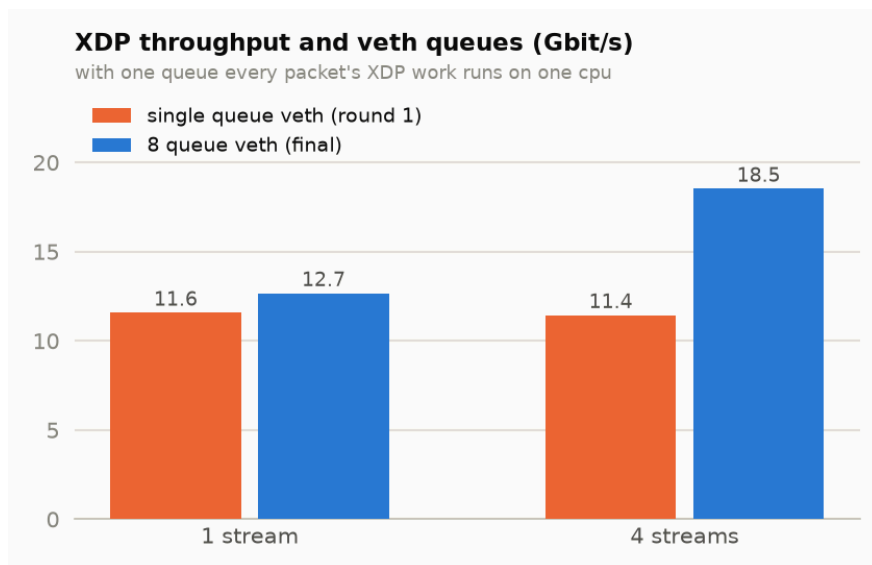


Figure 4. XDP throughput with single-queue and eight-queue veths.

Reliability. The smoke test passes for all three modes. In the test we use for the demo, a 20-second iperf3 transfer runs through the VIP while the backend it landed on is removed from the ring: the transfer continues at 12.9 Gbit/s without a dip, because its packets are served from the connection table, while new requests all go to the remaining backend; adding the backend back restores the split.

Caveats. Everything shares one machine, veth is not a NIC (extra copies, no hardware RSS, reordering), and a laptop throttles when hot. The ranking and the per-packet costs were stable across four benchmark rounds; the absolute gigabit figures are testbed numbers, and on real hardware the XDP advantage should be larger because the copies disappear.

6 Problems we ran into

Most of the difficulty was not in the load balancer itself.

- **XDP_TX into a veth went nowhere.** The first end-to-end test timed out although the program's counters said the packets were sent; `ethtool -S` showed `xdp_tx_errors`. veth puts the frame into the peer's receive ring, which only exists when the peer has an XDP program attached. A do-nothing program on the peer fixed it.
- **Round-robin broke iperf3**, which opens a control and a data connection. This produced the hash ring and the flag to hash without the client port.
- **Retransmissions without drops.** Multi-stream iperf3 showed tens of thousands of retransmissions. The backend's out-of-order counters were huge and the same test with the load balancer removed reordered just as much: the veth path reorders, the program was innocent.
- **The raw forwarder stopped answering** after a busy run. A capture showed the backend's SYN-ACK arriving and being dropped: one entry of a connection's pair had been evicted from the table, the other kept matching the reused client port, and the missing one was never re-created. The XDP program's LRU map had the same hole in principle. Hence the rule that a SYN never consults the table.
- **The benchmark harness had bugs of its own.** The test HTTP server spun at 100% CPU after a client closed without sending (a stale `errno` check), which heated the laptop and inflated every CPU number until it was found; two modes run half an hour apart differed by 45 μ s of latency from temperature alone; and reused client ports collided in the backends' `TIME_WAIT` sockets between the no-LB and the VIP runs because Linux picks TCP timestamp clocks per address pair. The script now pins the governor, interleaves the modes, and uses a separate client address for the no-LB runs.

7 Future work and conclusion

The natural next steps are the parts of Katran we left out: UDP, IPv6, IP-in-IP encapsulation with direct server return so the load balancer sees only requests, Maglev hashing, and backend health checks. The proxy baseline is single-threaded; a multi-process one would be a fairer throughput comparison, although the per-packet CPU cost, which is the comparison we trust most, would not change in its favour. Everything should also be re-measured on a physical NIC.

We built a layer-4 load balancer that runs inside the kernel at the XDP hook, with a user-space control plane talking to it through maps, and two conventional load balancers to compare it with. On the same testbed the XDP version forwards more traffic, sets up more connections per second, adds a third of the latency of a

proxy and costs about half the CPU per gigabyte. Along the way we learned how the verifier shapes a program, how kernel and user space share state through maps, how to rewrite packets and fix checksums without touching the payload, and that benchmarking a load balancer honestly means first finding out what the testbed itself is doing.

References

1. N. Shirokov and R. Dasineni, "Open-sourcing Katran, a scalable network load balancer," Facebook Engineering, May 2018.
2. T. Høiland-Jørgensen et al., "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel," ACM CoNEXT, 2018.
3. D. E. Eisenbud et al., "Maglev: A Fast and Reliable Software Network Load Balancer," USENIX NSDI, 2016.
4. Cilium project, "BPF and XDP Reference Guide," docs.cilium.io.
5. A. Rijssinghani, "Computation of the Internet Checksum via Incremental Update," RFC 1624, 1994.
6. Katran source code, github.com/facebookincubator/katran; XDP tutorial, github.com/xdp-project/xdp-tutorial.